Nome: Lucas Lima de Barros Lica. **Turma:** 2°AMS.

4. Atividade 1 — Analisando o problema

4.1 - Qual é a responsabilidade da classe CalculadoraDesconto?

R: Na primeira implementação, a classe CalculadoraDesconto é responsável por calcular o valor final da compra de acordo com o tipo de cliente informado. Para isso, ela verifica qual é o tipo de cliente e aplica diretamente a regra de desconto correspondente por meio de estruturas condicionais, como if.

Por exemplo, a classe verifica se o cliente é COMUM, VIP ou PREMIUM e, para cada caso, aplica um desconto diferente. Portanto, além de realizar o cálculo, ela também precisa conhecer todas as regras de desconto existentes.

Esse é justamente o problema dessa abordagem: a classe acaba acumulando duas responsabilidades. Ela calcula o desconto e, ao mesmo tempo, administra quais regras devem ser aplicadas. Dessa forma, quanto mais tipos de desconto forem adicionados, maior ficará a responsabilidade da CalculadoraDesconto.

4.2 - Quais comportamentos podem variar?

R: Os comportamentos que podem variar são as regras de cálculo do desconto, ou seja, a forma como o valor da compra será alterado de acordo com cada tipo de cliente ou modalidade de desconto.

Na implementação inicial, já existem três comportamentos diferentes:

- Cliente comum → 5% de desconto;
- Cliente VIP → 10% de desconto;
- Cliente Premium → 20% de desconto.

Além desses, o próprio exercício apresenta outras possibilidades, como desconto de aniversário de 15%, desconto promocional de 25%, desconto para funcionários de 30% e desconto especial de Black Friday de 40%.

Portanto, o comportamento que varia é o algoritmo utilizado para calcular o valor final da compra. O processo de receber um valor e calcular o preço continua existindo, mas a fórmula utilizada depende da estratégia de desconto escolhida.

4.3 - O que aconteceria se fossem criados 10 novos tipos de desconto?

R: Se fossem criados 10 novos tipos de desconto, a classe CalculadoraDesconto precisaria receber mais 10 condições para verificar cada nova regra. Isso faria o código crescer e ficar cada vez mais concentrado em uma única classe.

O próprio enunciado destaca esse problema ao imaginar a inclusão de descontos de aniversário, promocionais, para funcionários e de Black Friday, observando que o código começaria a acumular vários if/else.

Além de aumentar o tamanho da classe, isso deixaria a CalculadoraDesconto cada vez mais difícil de entender e modificar. Uma mudança em qualquer regra exigiria editar a mesma classe, aumentando também o risco de introduzir erros em regras que já estavam funcionando.

4.4 – Seria necessário modificar a classe CalculadoraDesconto?

R: Sim. Na primeira implementação, seria necessário modificar a CalculadoraDesconto.

Como todas as regras estão diretamente dentro dela, a criação de um novo tipo de desconto exigiria adicionar uma nova condição e o respectivo cálculo.

Por exemplo, para adicionar um desconto de funcionário, seria necessário incluir mais uma verificação no código da calculadora. Isso significa que a classe principal ficaria diretamente dependente de cada nova regra criada.

Esse é justamente o problema que o padrão Strategy procura resolver. Na solução proposta posteriormente pelo laboratório, cada regra passa a ser uma estratégia independente, permitindo adicionar novos comportamentos sem alterar CalculadoraDesconto. Portanto: Primeira implementação: Nova regra → modificar CalculadoraDesconto ; Com Strategy: Nova regra → criar uma nova estratégia.

4.5 – Você considera essa solução fácil de manter? Justifique.

R: Não considero a primeira implementação uma solução fácil de manter, principalmente quando o sistema começa a receber novos tipos de desconto.

Para poucos tipos de desconto, o código pode parecer simples e funcionar corretamente. Entretanto, conforme surgem novas regras, a CalculadoraDesconto passa a acumular cada vez mais condições e responsabilidades. O próprio laboratório apresenta essa situação como um problema, pois novos descontos fariam o código acumular vários if/else.

Isso dificulta a manutenção por alguns motivos. Primeiro, a classe fica maior e mais complexa. Segundo, qualquer alteração em uma regra exige modificar a mesma classe que contém todas as outras regras. Terceiro, quanto maior a quantidade de condições, maior a possibilidade de um erro em uma alteração afetar algum comportamento que já estava funcionando.

Por isso, embora a solução inicial seja funcional para um sistema pequeno, ela não é adequada para um sistema que precisa crescer e receber frequentemente novas regras de desconto.

Essa dificuldade é justamente a motivação para utilizar o padrão Strategy: separar cada comportamento variável em uma classe própria, permitindo que novas regras sejam adicionadas de forma mais organizada e sem modificar continuamente a classe principal.

5. Atividade 2 — Modelagem UML

Antes de programar, construa um Diagrama de Classes UML para a solução proposta inicialmente. Seu diagrama deve representar pelo menos:

• CalculadoraDesconto

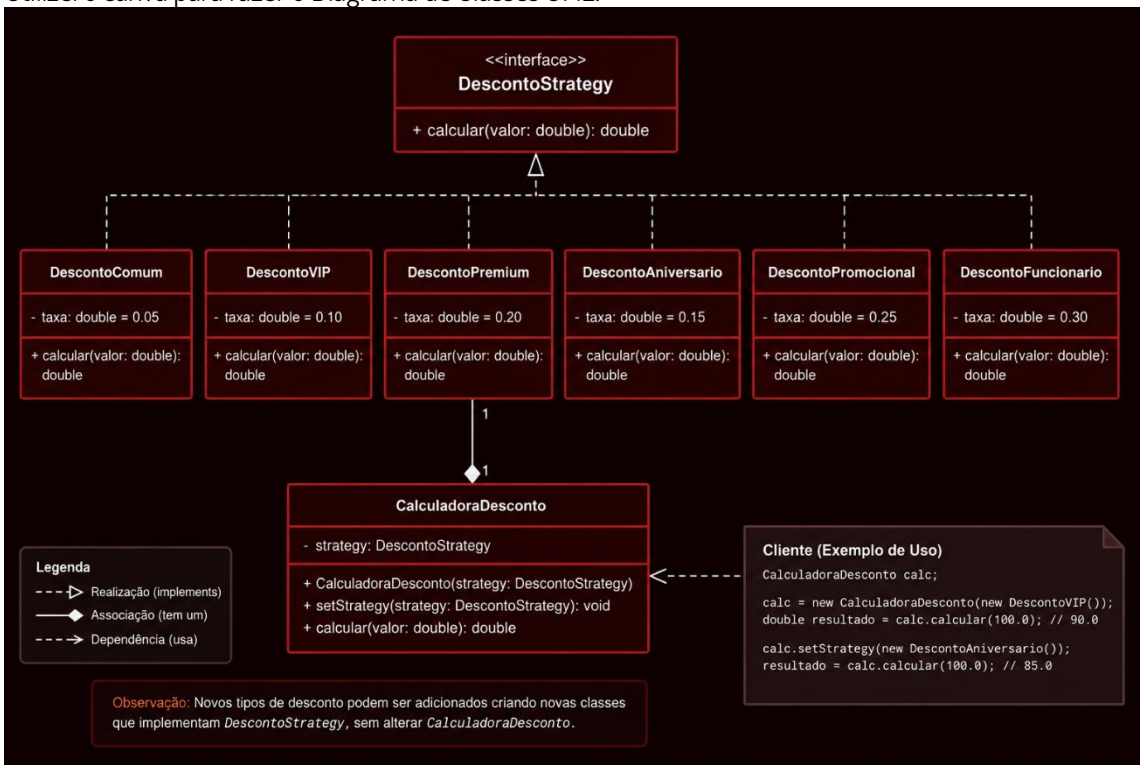
e o método:

• calcular()

Desafio - Represente também as diferentes regras de desconto.

Você pode utilizar uma ferramenta como: Draw.io; Lucidchart; PlantUML; StarUML; Visual Paradigm; ou outra ferramenta UML disponível.

Utilizei o canva para fazer o Diagrama de Classes UML.



16. Reflexão sobre o projeto

Questão 1 - Qual era o principal problema da primeira implementação?

R: O principal problema da primeira implementação era que a classe CalculadoraDesconto ficava responsável por conhecer e controlar todas as regras de desconto. Para isso, ela utilizava vários if/else verificando o tipo de cliente, como COMUM, VIP e PREMIUM.

Esse modelo funciona quando existem poucas regras, mas começa a se tornar problemático à medida que o sistema cresce. A própria atividade apresenta a possibilidade de surgirem descontos de

aniversário, promocionais, para funcionários e de Black Friday, fazendo com que a classe acumule cada vez mais condições.

O problema, portanto, não é apenas a quantidade de if/else, mas principalmente a mistura de várias regras de negócio dentro de uma única classe. Dessa forma, toda vez que uma nova regra fosse criada, seria necessário alterar CalculadoraDesconto.

Isso torna o código mais difícil de manter, testar e evoluir. Uma alteração em uma regra existente também poderia afetar outras partes da classe.

Questão 2 - O que representa a interface DescontoStrategy?

R: A interface DescontoStrategy representa um contrato para o comportamento de cálculo do desconto**.** Ela define que toda estratégia de desconto deve possuir um método calcular(valor).

No laboratório, a interface é apresentada como:

public interface DescontoStrategy { double calcular(double valor); }

Isso significa que a interface não precisa saber como cada desconto será calculado. Ela apenas estabelece o que todas as estratégias devem ser capazes de fazer**.**

Por exemplo:

DescontoComum

↓

calcular()

DescontoVIP

↓

calcular()

DescontoPremium

↓

calcular()
Cada classe implementa o mesmo contrato, mas possui sua própria regra.

Essa estrutura permite que CalculadoraDesconto trabalhe apenas com DescontoStrategy, sem precisar conhecer os detalhes de cada estratégia. O laboratório destaca justamente que CalculadoraDesconto trabalha com a abstração DescontoStrategy, permitindo trocar a estratégia utilizada.

Questão 3 - Qual é a responsabilidade de CalculadoraDesconto?

R: A responsabilidade de CalculadoraDesconto é utilizar uma estratégia de desconto para realizar o cálculo**,** sem conhecer a implementação específica daquela estratégia.

Na solução apresentada pelo laboratório, a classe recebe um DescontoStrategy:

```
public class CalculadoraDesconto {
   private DescontoStrategy strategy;
   public CalculadoraDesconto(DescontoStrategy strategy) {
      this.strategy = strategy;
   }
   public double calcular(double valor) {
      return strategy.calcular(valor);
   }
}
```

Ela simplesmente possui uma estratégia e delega a ela o cálculo.

Por exemplo:

CalculadoraDesconto

↓

DescontoVIP

↓

calcular(100)

↓

90

Depois podemos trocar a estratégia:

CalculadoraDesconto

↓

DescontoPremium

↓

calcular(100)

↓

80

Portanto, CalculadoraDesconto funciona como o Contexto do padrão Strategy. Sua responsabilidade é coordenar o uso da estratégia, e não implementar todas as regras de desconto.

Questão 4 – Qual é a responsabilidade de DescontoVIP?

R: A responsabilidade de DescontoVIP é implementar exclusivamente a regra de desconto destinada aos clientes VIP**.** No laboratório, clientes VIP recebem 10% de desconto**.** Dessa forma, para uma compra de R$ 100,00:

Valor original: R$ 100,00

Desconto: 10%

Valor final: R$ 90,00

A implementação fornecida é:

```
public class DescontoVIP implements DescontoStrategy {
  @Override
  public double calcular(double valor) {
    return valor * 0.90;
  }
}
```

O ponto importante é que DescontoVIP possui uma responsabilidade muito específica: saber calcular o desconto VIP. Ela não precisa conhecer as regras de DescontoComum, DescontoPremium ou qualquer outra modalidade, isso faz com que a classe seja simples, independente e fácil de testar.

Questão 5 – Por que podemos criar DescontoAniversario sem modificar CalculadoraDesconto?

R: Podemos criar DescontoAniversario sem modificar CalculadoraDesconto porque ambas dependem de uma abstração comum, que é DescontoStrategy. A nova classe simplesmente implementa essa interface:

```
DescontoStrategy
    ▲
    |
    └── DescontoAniversario
```

O laboratório determina explicitamente que CalculadoraDesconto não pode ser modificada para adicionar essa nova regra, a utilização pode ser feita assim:

```
calculadora = new CalculadoraDesconto(
  new DescontoAniversario()
);
```

Para uma compra de R$ 100,00, o resultado esperado é R$ 85,00, pois o desconto é de 15%.

Isso é possível porque CalculadoraDesconto não precisa saber que existe uma classe chamada DescontoAniversario. Ela só precisa saber que recebeu alguma coisa que implementa DescontoStrategy. Se tornando uma das principais vantagens do padrão Strategy: novos comportamentos podem ser adicionados criando novas estratégias, sem alterar o contexto existente.

Questão 6 - Qual é a vantagem de separar cada regra de desconto em uma classe?

R: A principal vantagem é separar responsabilidades, fazendo com que cada classe tenha apenas uma regra de negócio para controlar.

Por exemplo:

DescontoComum → 5%

DescontoVIP → 10%

DescontoPremium → 20%

DescontoAniversario → 15%

DescontoPromocional → 25%

DescontoFuncionario → 30%

Essas são estratégias independentes apresentadas no laboratório, essa separação traz várias vantagens. Primeiro, o código fica mais fácil de entender. Ao abrir DescontoVIP, por exemplo, encontramos somente a regra de desconto VIP, segundo, cada regra pode ser testada individualmente. Podemos testar DescontoVIP sem precisar testar todas as outras estratégias ao mesmo tempo, terceiro, uma alteração em uma regra não exige modificar as outras (se o desconto VIP mudar de 10% para 12%, somente a estratégia correspondente precisa ser alterada), e quarto novas regras podem ser acrescentadas com menor impacto no código existente. A atividade demonstra exatamente isso ao adicionar DescontoAniversario, DescontoPromocional e DescontoFuncionario.

Questão 7 - Qual princípio de projeto está relacionado a essa solução?

R: O princípio de projeto relacionado diretamente a essa solução é o Open/Closed Principle (OCP), ou Princípio Aberto/Fechado, na qual o próprio laboratório estabelece a relação entre o padrão Strategy e o OCP. A ideia do OCP é que uma classe ou módulo deve estar aberto para extensão, mas fechado para modificação, No caso da CalculadoraDesconto pode receber novos comportamentos sem precisar ser modificada. Por exemplo, inicialmente temos:

DescontoComum, DescontoVIP e DescontoPremium,

Depois podemos adicionar:

DescontoAniversario, DescontoPromocional e DescontoFuncionario

Sem alterar o código da CalculadoraDesconto, desde que essas novas classes implementem DescontoStrategy. Isso demonstra exatamente o princípio:

NOVA REGRA

↓

Nova classe

↓

Implementa DescontoStrategy

↓

Sistema continua funcionando

Em vez de:

NOVA REGRA

↓

Modificar CalculadoraDesconto

↓

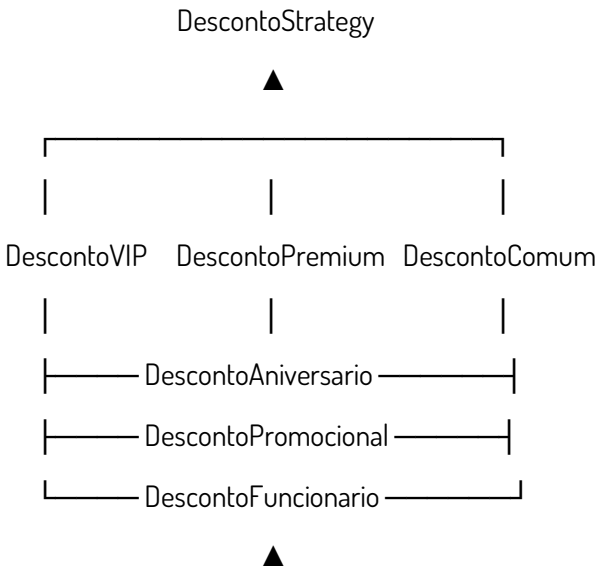
Adicionar mais if/else

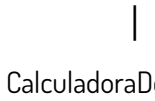
↓

Aumentar a complexidade

Questão 8 – Em suas próprias palavras, explique o que é o padrão Strategy.

R: O padrão Strategy é uma forma de organizar o programa quando existe uma determinada operação que pode ser realizada de várias maneiras diferentes. Em vez de colocar todas essas maneiras dentro de uma única classe usando vários if/else, cada comportamento é colocado em uma classe separada. Todas essas classes seguem um mesmo contrato, representado pela DescontoStrategy. No projeto feito no word ficou, assim:





A CalculadoraDesconto não precisa saber como cada desconto funciona. Ela apenas recebe uma estratégia e solicita que ela faça o cálculo. Por exemplo:

Calculadora + DescontoVIP

↓

10% de desconto

Calculadora + DescontoPremium

↓

20% de desconto

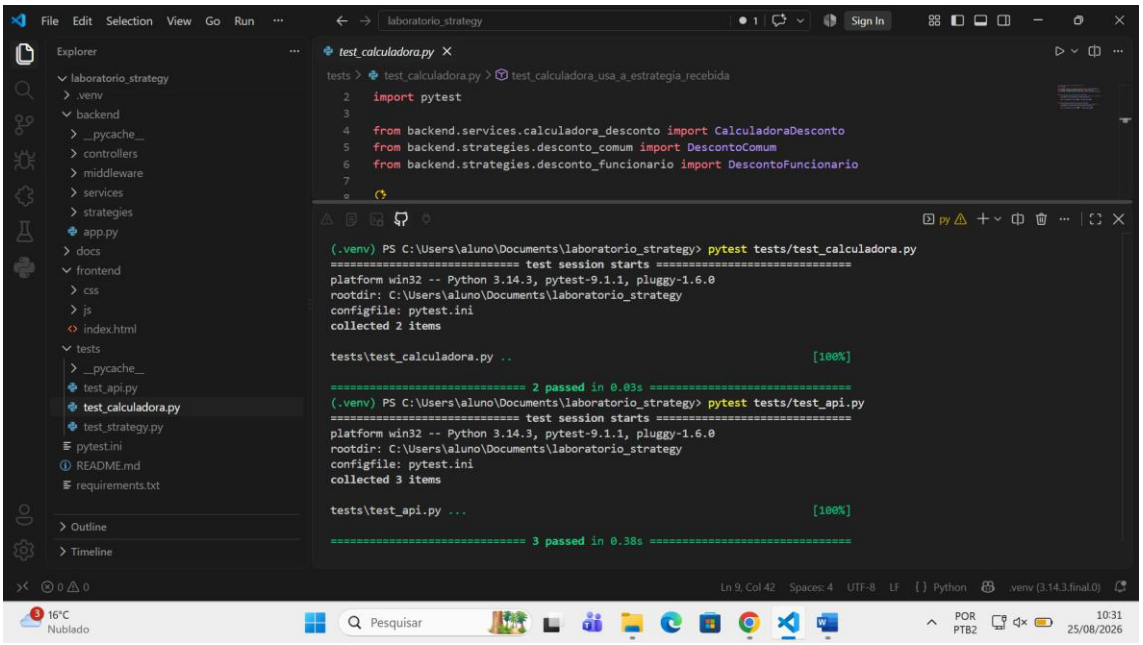
Calculadora + DescontoAniversario

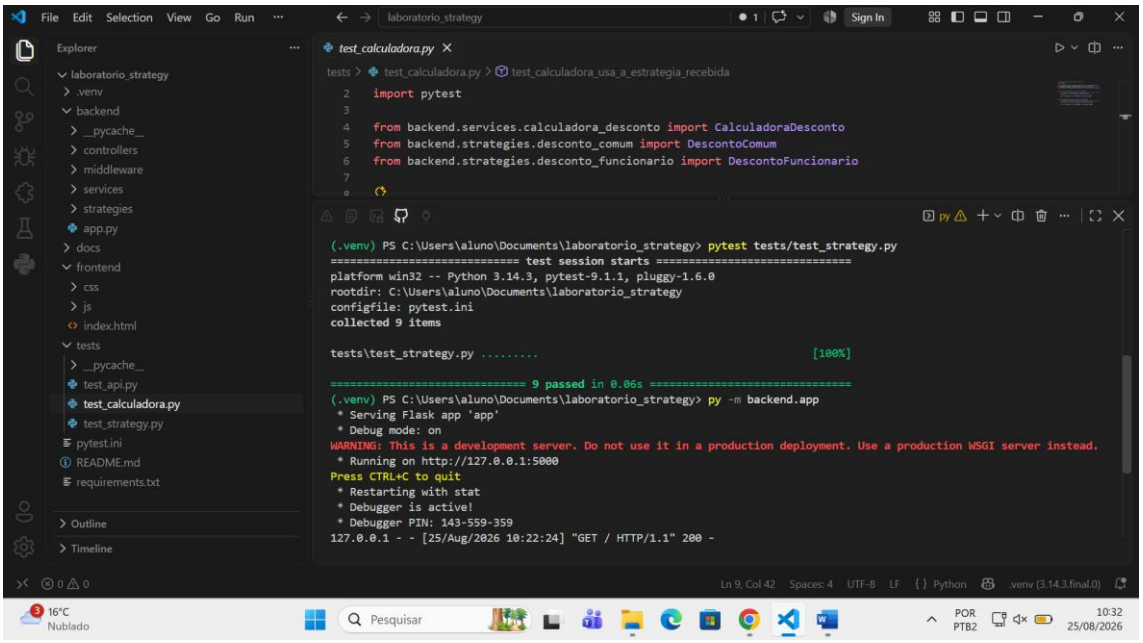
↓

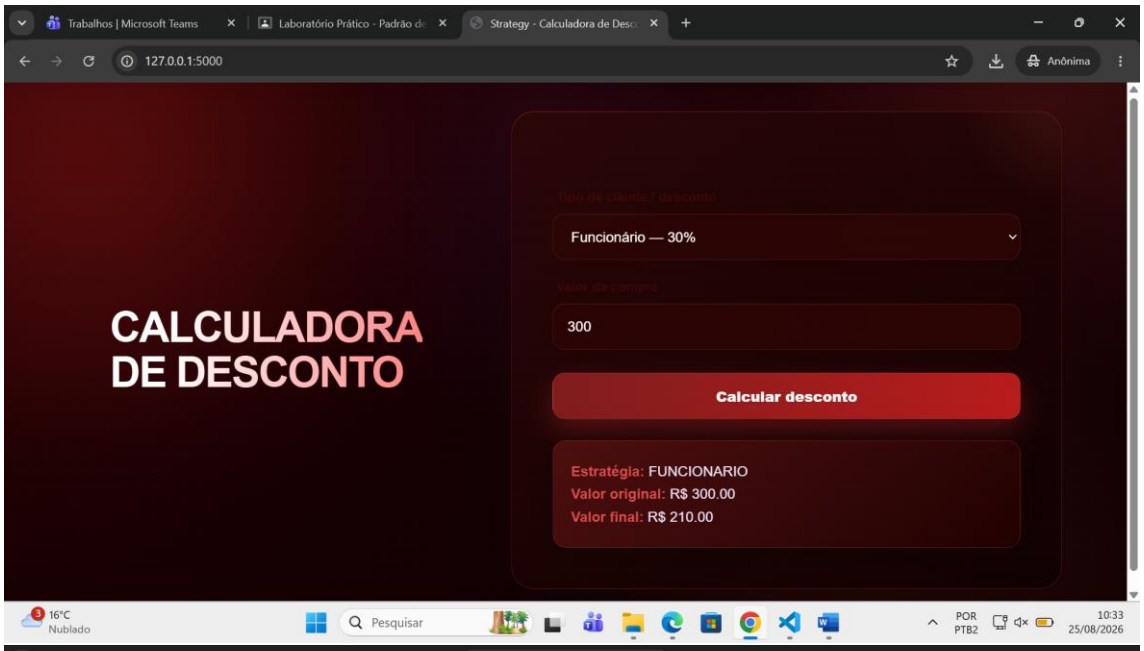
15% de desconto

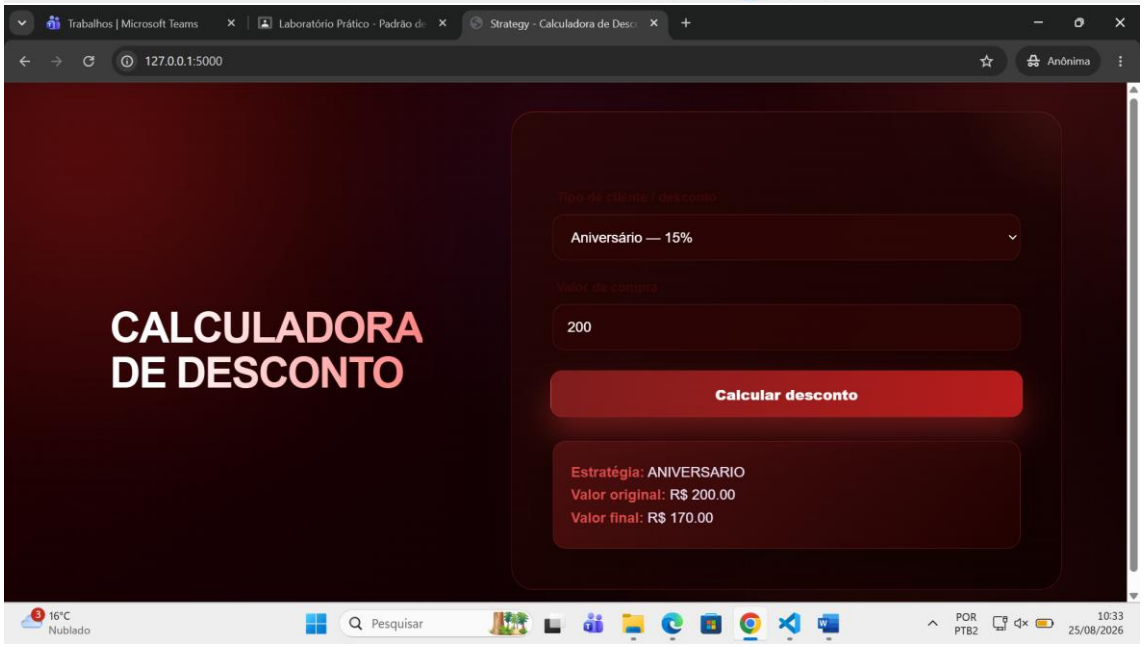
Assim, as estratégias podem ser trocadas sem alterar a estrutura principal da calculadora, na minha compreensão, essa é a principal ideia do Strategy: separar comportamentos que podem variar em classes independentes, permitindo que esses comportamentos sejam trocados conforme a necessidade, sem colocar todas as regras dentro de uma única classe, isso também explica por que o laboratório termina perguntando o que aconteceria caso fossem criados 50 novos tipos de desconto: a intenção é perceber que, usando Strategy, podemos criar novas estratégias sem transformar CalculadoraDesconto em uma enorme sequência de condições.

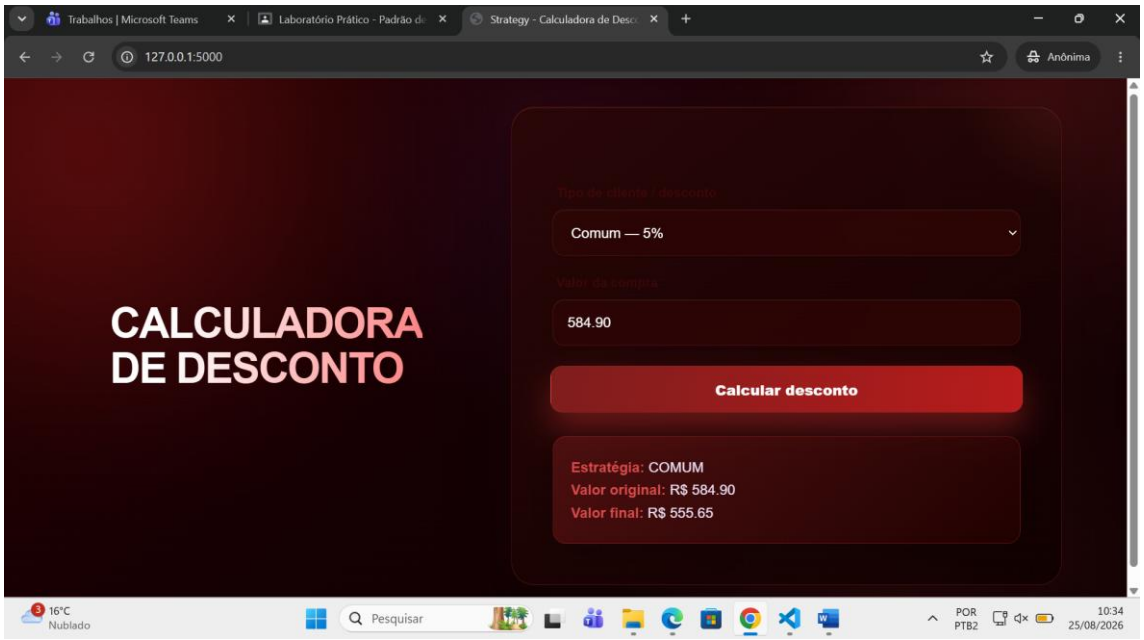
Prints de execução

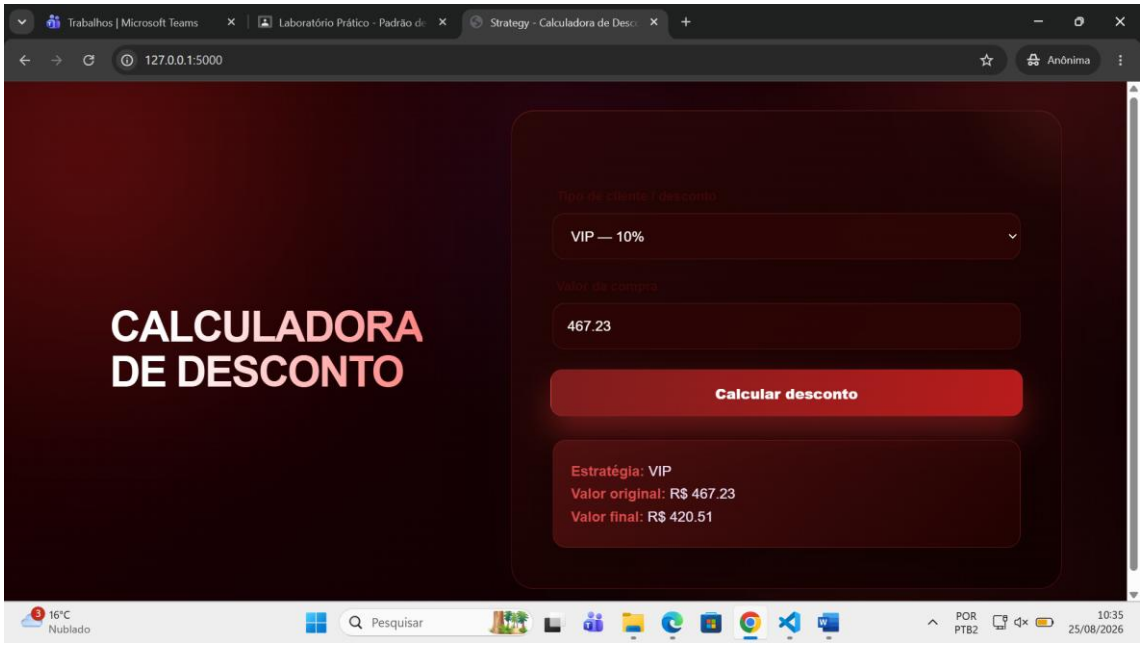

File Edit Selection View Go Run
laboratorio_strategy
Sign In
Explorer
test_calculadora.py
tests > test_calculadora.py > test_calculadora_usa_a_estrategia_recebida
2 import pytest
4 from backend.services.calculadora_desconto import CalculadoraDesconto
5 from backend.strategies.desconto_comum import DescontoComum
6 from backend.strategies.desconto_funcionario import DescontoFuncionario
laboratorio_strategy
.venv
backend
__pycache__
controllers
middleware
services
strategies
app.py
docs
frontend
css
js
index.html
tests
__pycache__
test_api.py
test_calculadora.py
test_strategy.py
pytest.ini
README.md
requirements.txt
Outline
Timeline
(.venv) PS C:\Users\aluno\Documents\laboratorio_strategy> pytest tests/test_calculadora.py
test session starts
platform win32 -- Python 3.14.3, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Users\aluno\Documents\laboratorio_strategy
configfile: pytest.ini
collected 2 items
tests\test_calculadora.py .. [100%]
2 passed in 0.03s
(.venv) PS C:\Users\aluno\Documents\laboratorio_strategy> pytest tests/test_api.py
test session starts
platform win32 -- Python 3.14.3, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Users\aluno\Documents\laboratorio_strategy
configfile: pytest.ini
collected 3 items
tests\test_api.py ... [100%]
3 passed in 0.38s
Ln 9, Col 42 Spaces: 4 UTF-8 LF Python .venv (3.14.3.final.0)
16°C
Nublado
Pesquisar
POR
PTB2
10:31
25/08/2026


File Edit Selection View Go Run
laboratorio_strategy
Sign In
Explorer
test_calculadora.py
tests > test_calculadora.py > test_calculadora_usa_a_estrategia_recebida
2 import pytest
4 from backend.services.calculadora_desconto import CalculadoraDesconto
5 from backend.strategies.desconto_comum import DescontoComum
6 from backend.strategies.desconto_funcionario import DescontoFuncionario
laboratorio_strategy
.venv
backend
__pycache__
controllers
middleware
services
strategies
app.py
docs
frontend
css
js
index.html
tests
__pycache__
test_api.py
test_calculadora.py
test_strategy.py
pytest.ini
README.md
requirements.txt
Outline
Timeline
(.venv) PS C:\Users\aluno\Documents\laboratorio_strategy> pytest tests/test_strategy.py
test session starts
platform win32 -- Python 3.14.3, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Users\aluno\Documents\laboratorio_strategy
configfile: pytest.ini
collected 9 items
tests\test_strategy.py ..... [100%]
9 passed in 0.06s
(.venv) PS C:\Users\aluno\Documents\laboratorio_strategy> py -m backend.app
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 143-559-359
127.0.0.1 - - [25/Aug/2026 10:22:24] "GET / HTTP/1.1" 200 -
Ln 9, Col 42 Spaces: 4 UTF-8 LF Python .venv (3.14.3.final.0)
16°C
Nublado
Pesquisar
POR
PTB2
10:32
25/08/2026


Trabalhos | Microsoft Teams
Laboratório Prático - Padrão de
Strategy - Calculadora de Desc
127.0.0.1:5000
Anônima
CALCULADORA
DE DESCONTO
Funcionário — 30%
300
Calcular desconto
Estratégia: FUNCIONARIO
Valor original: R$ 300.00
Valor final: R$ 210.00
16°C
Nublado
Pesquisar
POR
PTB2
10:33
25/08/2026


Trabalhos | Microsoft Teams
Laboratório Prático - Padrão de
Strategy - Calculadora de Desc
127.0.0.1:5000
Anônima
CALCULADORA
DE DESCONTO
Aniversário — 15%
200
Calcular desconto
Estratégia: ANIVERSARIO
Valor original: R$ 200.00
Valor final: R$ 170.00
16°C
Nublado
Pesquisar
POR
PTB2
10:33
25/08/2026


Trabalhos | Microsoft Teams
Laboratório Prático - Padrão de
Strategy - Calculadora de Desc
127.0.0.1:5000
Anônima
CALCULADORA
DE DESCONTO
Tipo de cliente / desconto
Comum — 5%
Valor da compra
584.90
Calcular desconto
Estratégia: COMUM
Valor original: R$ 584.90
Valor final: R$ 555.65
16°C
Nublado
Pesquisar
POR
PTB2
10:34
25/08/2026


Trabalhos | Microsoft Teams
Laboratório Prático - Padrão de
Strategy - Calculadora de Desc
127.0.0.1:5000
Anônima
CALCULADORA
DE DESCONTO
Tipo de cliente / desconto
VIP — 10%
Valor da compra
467.23
Calcular desconto
Estratégia: VIP
Valor original: R$ 467.23
Valor final: R$ 420.51
16°C
Nublado
Pesquisar
POR
PTB2
10:35
25/08/2026